

25

Deep Neural Networks for Grasp Planning in Domestic Service Robots

German Alday-Salazar

School of Engineering-National Autonomous University of Mexico

Itzel González-Jiménez

School of Engineering-National Autonomous University of Mexico

Marco Negrete

School of Engineering-National Autonomous University of Mexico

CONTENTS

25.1	Introduction	450
25.2	Related work	450
25.3	Problem statement and nature of data	451
25.4	Geometric approach for generating grasp proposals	452
25.5	Deep Neural Network architecture	453
25.5.1	Convolutional Encoder	454
25.5.2	Position network	455
25.5.3	Orientation network	456
25.5.3.1	Geodesic regression approach	456
25.5.3.2	Kernel mixture model approach	457
25.6	Dataset generation	458
25.6.1	Geometric constraints	458
25.6.2	Manually generated dataset	459
25.6.3	Sampling based dataset	459
25.7	Experiments	460
25.7.1	Testbed	460
25.8	Results	461
25.9	Conclusion	461

Object manipulation is a key skill in domestic service robots that has been addressed using analytical and statistical methods, and more recently, with machine learning techniques. In this work, we address the problem of proposing a suitable final effector pose to grasp an object given an object's partial

point cloud, i.e., we assume that there is no full view of the object. We propose a DNN architecture composed of two main parts: a stage for getting the final effector position and another one, to get the orientation. To propose orientations, we used two approaches: geodesic regression and a kernel mixture model. We tested our proposal in simulation using Gazebo and ROS and we compared performance with our previous grasping system, which is based on Principal Component Analysis and heuristic rules based on object's shape. Results show that our proposal has an average accuracy of around 65%, which is similar to the previous geometric-based system. Nevertheless, the previous system strongly relies on object segmentation and it is very sensitive to outliers in the object's point cloud, which makes our proposal a more robust and reliable solution.

25.1 Introduction

Object manipulation and real world interaction is an open problem in the field of autonomous robots. As domestic service robots became popular, many research studies on object manipulation based on computer vision emerged. Currently, there is extensive research in the field of robotic grasping proposal and optimization. RGB-D cameras also became popular in mobile robots due to the significant lower price and the availability of out-of-the-box libraries for depth and color image registration. The problem of planning movements for grasping has been tackled using analytical and statistical methods and, more recently, with machine learning techniques. Grasping an object implies getting a feasible final effector configuration (a position p and an orientation q), planning a trajectory to reach such pose while avoiding obstacles, and finally, solving the inverse kinematics and executing a control law to move the manipulator.

25.2 Related work

Before the adoption of neural network-based approaches, research relied on a combination of analytical and statistical methods. Partial point clouds of objects were analyzed using Principal Component Analysis (PCA) to identify the principal axis of the object. Grasp candidates were then generated along this axis, and simple geometric rules were applied to determine the optimal contact points for grasping[819],[820],[821].

In recent years, neural network-based learning methods have gained significant relevance due to their ability to learn object grasps directly from

samples with high success rates. Deep learning models such as Sparse-GRConvNet[822], which integrate visual information to infer optimal grasping positions, have achieved an accuracy of 97.75% on datasets such as the Cornell Grasping Dataset(CGD). Some works incorporate neural networks for object segmentation and recognition, such as Mask R-CNN [823]. Architectures like Dex-Net 2.0 propose deep learning systems for grasp planning based on a depth image and a candidate grasp pose. They use synthetic point clouds to avoid the high costs of collecting real-world data, achieving success rates that surpass those obtained using learning methods based solely on real data [824]. Novel architectures such as OS-TOG (One-Shot Task-Oriented Grasping) enable robots to generate task-oriented grasps using one-shot learning per object, achieving a task-oriented grasp accuracy of 83.2 percent in both simulation and real-world grasping scenarios [825].

On the other hand, reinforcement learning-based architectures like Grasp-Q-Network develop a grasping policy that does not rely on a predefined set of grasp samples but instead learns autonomously through interaction with the environment, using visuomotor feedback [826]. Its reward function assigns positive values to successful grasps, partial rewards for contact with the object, and penalties for inaction, thus encouraging efficient behaviors.

To improve robustness against environmental noise, some works have developed deep residual shrinkage networks (DSRN) [827], based on ResNet, which enable the system to learn to ignore irrelevant parts of the image, such as background or object regions that are not useful for grasping.

Convolutional Neural Networks (CNNs) have gained popularity due to their strong ability to extract important features from images for robotic grasp planning. Some approaches operate in real time using direct regression and classification models to predict optimal grasp positions and the object category [828]. In contrast, GR-ConvNet v2 is an architecture that requires neither classification nor semantic segmentation. It uses an inference module to generate maps indicating viable grasp regions for each pixel based on the local geometry of the RGB-D image, allowing the system to generalize grasping capabilities to previously unseen objects [829].

Sensor information can often be incomplete or noisy. This uncertainty has been addressed with probabilistic approaches that integrate Gaussian Mixture Models (GMMs) and discrete Bayesian networks to represent and model robotic grasping tasks. This enables the robot to reason about task requirements, facilitating task-oriented grasp prediction and object selection [830]. Probabilistic models are also used to give robots the ability to grasp objects that are partially occluded or deformed. By probabilistically modeling the geometry of objects, robots can infer complete shapes and plan effective grasps[831].

25.3 Problem statement and nature of data

We seek to construct a neural network architecture capable of generating suitable grasp proposals directly from the point cloud acquired from an RGB-D camera mounted at the top of the robot's head.

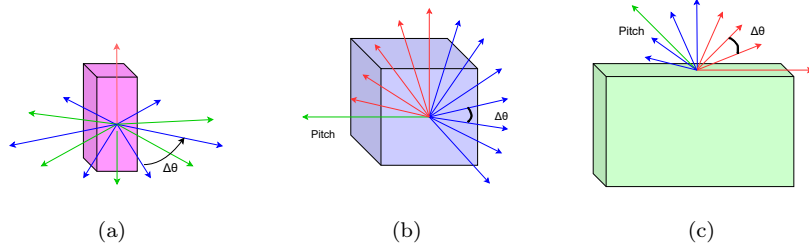
There are some important considerations about the nature of the problem and the representation of data:

1. There are multiple ways to represent the group of 3D rotations $\mathbf{SO}(3)$. We chose unitary quaternions over rotation matrices because they can store the information in lower dimensionality and have a constraint that is much easier to enforce (unit norm vs unit determinant). Furthermore, quaternions do not experience the problem of gimbal lock sometimes present when using the Euler angles representation. However, quaternions come with their own set of problems. Mainly, that the data lie naturally within a manifold, and thus Euclidean metrics cannot be used. Additionally, the mapping of S^3 onto $SO(3)$ forms a double cover, that is, any rotation can be represented by the unitary quaternions q and $-q$.
2. For any given organized point cloud $c \in \mathbb{R}^{H \times W \times 3}$ that contains an object with an arbitrary position p_o and orientation q_o , there exists a set of multiple valid grasp candidates $g_1, g_2, \dots, g_n \in G$ within the robot configuration space.

25.4 Geometric approach for generating grasp proposals

Our previous system generates grasp proposals based on a geometric approach, and this system will be used as baseline for comparison against the proposed neural network-based approach.

As a first step, we use the RGB image to segment the object to be grasped using the YOLO library and contour detection. The resulting mask is used to get the object's point cloud c_o (subset of the cloud c given by the RGB-D camera). Then, we calculate the point cloud's covariance matrix and its eigenvalues $\lambda_1, \lambda_2, \lambda_3$ and eigenvectors E_1, E_2 , and E_3 , which approximate the directions of greatest variance among the point cloud. Since these vectors form a base that spans the cloud, they can be used as a coordinate system O_{obj} with origin at the object's centroid and adjusted to the object's shape. If we transform the object's cloud c_o into the new system O_{obj} , the distance between the minimum and maximum points on the main axes can be used

**FIGURE 25.1**

Grasp proposals for each type of geometry.

to create an oriented bounding box (OBB). The dimensions of this OBB are then used to classify the object into a set of basic geometric shapes as follows:

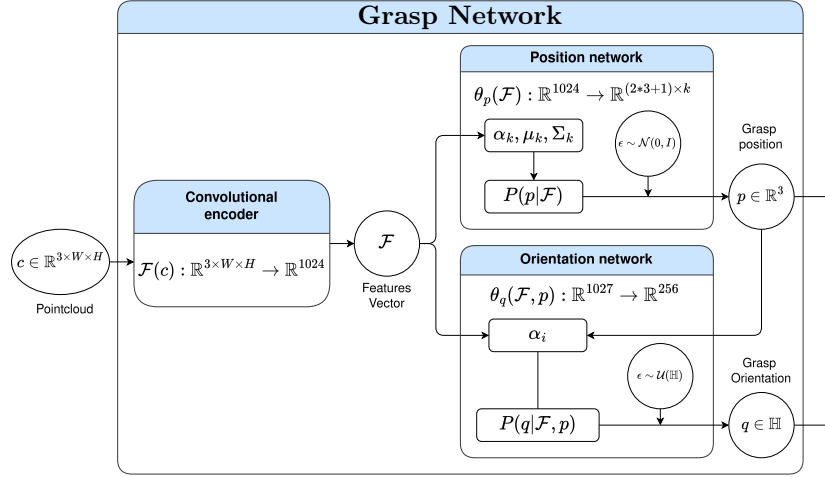
1. When $\lambda_1 \gg \lambda_2$ and $\lambda_2 \simeq \lambda_3$, the object can be seen as a prism and the manipulator should grasp it perpendicularly to E_1 (see Figure 25.1(a)).
2. When $\lambda_1 \simeq \lambda_2 \simeq \lambda_3$, the object can be seen as a cube and the manipulator should grasp it from top or front (see Figure 25.1(b)).
3. When $\lambda_1 \simeq \lambda_2$ and $\lambda_1 \gg \lambda_3$ the object can be seen as a flat box and the manipulator should grasp it perpendicularly to E_1 or E_2 with a grasping point located on the edge of the box (see Figure 25.1(c)).

In case 1, the grasp candidates are generated by roll rotations if E_1 is nearly normal to the surface, and with pitch rotations if E_1 is nearly parallel. Rotations are generated in a predefined interval of π radians with a predefined step $\Delta\theta$. In case 2, the grasp candidates are generated with pitch rotations from front to top positions and, in case 3, candidates are proposed such that E_1 is perpendicular to the gripper, and the grasp point is located at the edge of the box. Finally, for each grasping proposal, we solve the inverse kinematic and choose the configuration with the smallest joint angles.

25.5 Deep Neural Network architecture

In order to tackle the problem of finding suitable grasp proposals for object manipulation from vision sensors, a multi-step system based on deep learning was designed.

The core of the system is a deep neural network trained on a dataset

**FIGURE 25.2**

Neural Network system diagram

of multiple synthetic grasps created using the ROS-Gazebo simulated physics engine. This dataset contains the information of 100,000 point clouds recorded with 1,000,000 associated grasp proposals for objects with different geometries based on the Yale-CMU-Berkeley (YCB) object and model set.

The following system architecture has been proposed for the grasping neural network:

The system is subdivided in three main sections: Convolutional encoder, position network and orientation network.

25.5.1 Convolutional Encoder

The first part of the system is a deep convolutional neural network encoder consisting of multiple sequential layers created with Pytorch based on the original AlexNet architecture. The network takes as input a 200×200 3-channel depth image (x, y, z) and process it through an initial 2D-convolutional layer of 11×11 kernels with no padding and a stride of 1 with 32 output channels. The result is inserted into a batch normalization layer and consecutively into a (2,2) max-pool layer without padding.

Each following step consists of a convolution layer with decreasing kernel sizes but doubling output channels followed by a batch normalization layer and finally the ReLU activation function. The output channels of the final convolutional layer are then flattened and go through a fully connected layer until the final output returns a vector $\mathcal{F} \in \mathbb{R}^{1024}$ which represents the most important features of the point cloud.

The network is trained as an autoencoder using stochastic gradient descent

with $L2$ weight regularization to minimize the MSE between the points of the reconstructed input and the target.

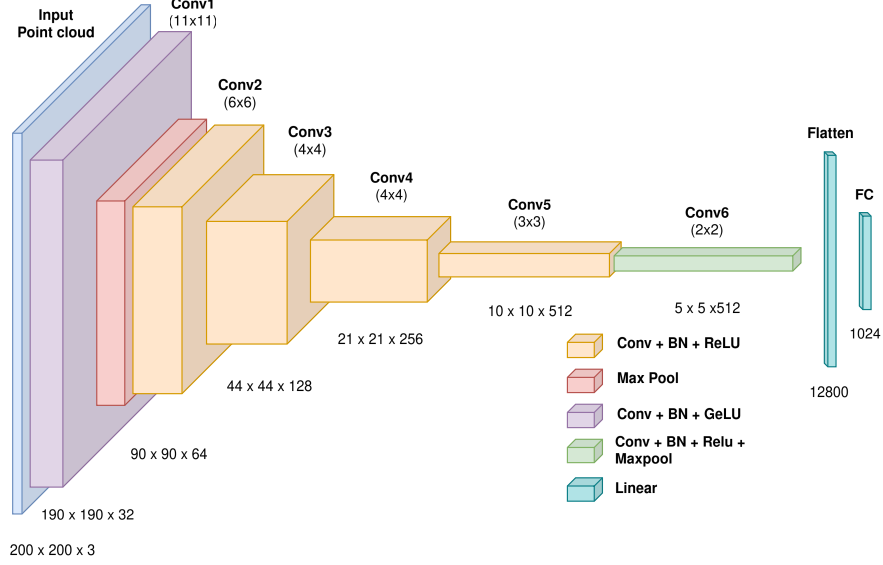


FIGURE 25.3
Convolutional Encoder architecture

25.5.2 Position network

Taking into consideration restriction 3, the conventional least squares regression model will not work for our problem because it is multi-valued (multi-modal) by nature. This method reduces to finding the conditional average of the target data conditioned on the input, which is not necessarily guaranteed to be a solution. In fact, we observed that our first model trained in such way converged to the trivial solution of proposing the center of the object since it was the point that had the least error to all the training grasps. Because of this, we opted to use a mixture density model [832] to predict the grasp candidate position. Instead of having as the network output the values of the position directly, we train the network to model the probability density function of the positions $P(p)$ conditioned on the input feature vector $\mathcal{F} \in \mathbb{R}^{1024}$ as a linear combination of kernel functions. We can then sample a position from this distribution.

$$P(p|\mathcal{F}) = \sum_{i=1}^m \alpha_i(\mathcal{F}) \phi_i(p|\mathcal{F}) \quad \sum_{i=1}^m \alpha_i = 1 \quad (25.1)$$

Where ϕ_i are distribution kernels defined on p , and α_i are the mixing

coefficients which represent the contribution of each kernel to the total density function. In order for it to be a valid probability distribution, the sum of the coefficients must be 1.

Since the value of the positions is defined over the 3-dimensional continuous Euclidean space and we assume the positions to be normally distributed, we select the Gaussian distribution as our choice of kernel.

$$\phi_i(p|\mathcal{F}) = \mathcal{N}(\mu_i(\mathcal{F}), \Sigma_i(\mathcal{F})) \quad (25.2)$$

As such, the outputs of the network are the $\alpha_k, \mu_k, \Sigma_k$ parameters of a linear combination of n Gaussian kernels as a function of the feature vector $\mathcal{F}$. The covariance matrices Σ_k are simplified to be the diagonal variance to reduce the complexity of the model.

$$P(p|x) = \sum_{k=1}^n \alpha_k(\mathcal{F}) \mathcal{N}(\mu_k(\mathcal{F}), \Sigma_k(\mathcal{F})) \quad (25.3)$$

The training objective then becomes maximum likelihood estimation. That is, to maximize the likelihood that the output model is the generator of the training data. Since the likelihood function of the parameters is positive monotonic, and we seek to optimize the parameters using stochastic gradient descent; we define our cost function as the negative log-likelihood loss as follows:

$$\mathcal{L}(p, \theta) = \prod_{i=1}^N P(p_i | \mathcal{F}_i; \theta) \quad (25.4)$$

$$-\log \mathcal{L}(p, \theta) = - \sum_{i=1}^N \log \left(\sum_{k=1}^n \alpha_k(\mathcal{F}_i; \theta) \mathcal{N}(p_i | \mu_k(\mathcal{F}_i; \theta), \Sigma_k(\mathcal{F}_i; \theta)) \right) \quad (25.5)$$

25.5.3 Orientation network

To obtain the orientation of the grasps, two network models with different approaches were proposed. Firstly as a geodesic regression model and finally as a kernel mixture model.

25.5.3.1 Geodesic regression approach

This network consists of a series of fully connected layers that take as input the concatenation of the point cloud feature vector from the convolutional encoder and a position, and returns a vector $u \in \mathbb{R}^4$

Because we are attempting to regress the orientation of the gripper as unitary quaternions, we must take into consideration the geometry of the space. We can't regress the output directly using MSE because Euclidean

metrics are not suitable for estimating distance in the manifold of quaternions. Furthermore with this approach we can't guarantee that the output vector is unitary. We attempted to solve this by simply normalizing the vector. However we found that this led to poor performance for our network.

To make sure that the network output is in the space of unitary quaternions, the hypersphere S^3 , we used principles of differential geometry to adapt our training algorithm. We modified our network to learn a vector u on the tangent space of the manifold $\mathcal{T}_b\mathcal{M}$ at the base point $b = (0, 0, 0, 1)$. Such that, when applying the exponential map $Exp(b, u)$ the result is guaranteed to lie on the manifold. Furthermore this allows vectors that are close together on the tangent space to be mapped nearby on the surface.

The training objective is to minimize the angle error between orientations and in this case amounts to minimizing the geodesic distance between the output and target quaternions. The loss function chosen was the squared geodesic distance.

$$E = d(q_o, q_t)^2 = \arccos(\langle q_o, q_t \rangle)^2 \quad (25.6)$$

Where $\langle \cdot, \cdot \rangle$ is the standard inner product in $\mathbb{R}^n$

25.5.3.2 Kernel mixture model approach

Similarly to the position network, the orientation network was defined to model the probability distribution of the orientations conditioned on the point cloud feature vector and the proposed position of the grasp $P(q|\mathcal{F}, p)$ using the kernel mixture density estimation approach.

This was done by approximating the conditioned probability distribution as a mixture of Von Mises-Fisher distributions over the 3-dimensional hypersphere. Thus, the network is trained to output the coefficients of the linear combination of the distributions as a function of the input vectors $\mathcal{F}, p$.

$$P(q) \approx \sum_i^n \alpha_i \mathcal{K}(q, q_i) \quad \sum_i^n \alpha_i = 1 \quad (25.7)$$

$$P(q|\mathcal{F}, p) \approx \sum_i^n \alpha_i(\mathcal{F}, p; \theta) \mathcal{K}(q, q_i) \quad \sum_i^n \alpha_i(\mathcal{F}, p; \theta) = 1 \quad (25.8)$$

The Von Mises-Fisher distribution is a unimodal probability distribution with isotropic covariance defined over the manifold of $(d - 1)$ -dimensional hyperspheres on $\mathbb{R}^d$. And it's described by the equation:

$$f_x(\mathbf{x}; \mu, \kappa) = C_p(\kappa) \exp(\kappa \mathbf{x}^T \mu) \quad (25.9)$$

Where the unitary vectors $\mathbf{x}, \mu \in \mathbb{R}^d$ are points on the space and the directional mean of the distribution respectively. And the concentration parameter

κ is a measure of spread (it defines how much the distribution clusters around the mean). The normalization constant is given by the following expression:

$$C_p(\kappa) = \frac{\kappa^{p/2-1}}{(2\pi)^{p/2} I_{p/2-1}(\kappa)} \quad (25.10)$$

Where I is the modified Bessel function of the first type.

In contrast to the position network, this network does not output all the parameters of the distributions μ, κ for their respective kernels. Instead, these parameters are fixed and are chosen a priori. The concentration parameter κ is fixed for all distributions and heuristically set to 320. The kernel centers $q_j = \mu_j$ are selected as the 256 points in the hyper-sphere that best describe the probability distribution for all quaternions in the dataset using the method of Competitive Learning Riemannian Quantization described in [833].

Similarly to the position network, the training objective is maximum likelihood estimation, and the negative log-likelihood chosen as the loss function.

25.6 Dataset generation

A data set $\mathcal{D} = \{\mathcal{C} = \{c_1, c_2, \dots, c_n\}, G = \{g_1, g_2, \dots, g_m\}\}$ was synthetically constructed from the simulation of our test environment on the ROS-Gazebo physics engine. Which contains a collection of $n = 100,000$ point clouds $c \in \mathbb{R}^{H \times W \times 3}$ and $m = 1,000,000$ associated successful grasp proposals $g = \{p, q | p \in \mathbb{R}^3, q \in \mathbb{H}_1\}$. Where p is the 3D position of the gripper base and q , the gripper orientation represented by a unit quaternion relative to the robot's base link frame.

The simulated scene was constructed by placing a single object with a random position and orientation on top of a desk with the robot standing in front of it. The objects were selected from a pool of 20 objects from the YCB object set.

Subsequently, a partial point cloud is acquired from the RGB-D camera sitting atop the head of the robot. All of the points are transformed to the reference frame of the robot's base link. The resulting point cloud is cut to a $200 \times 200 \times 3$ depth image centered around the object. If the point cloud does not contain the object, it is discarded and the simulation is reset.

25.6.1 Geometric constraints

There were several constraints to sampling implemented to reduce noise and ambiguity in the data.

Because of restriction 2, there are two valid quaternion representations for each orientation. So, in order to avoid redundancy we only sample those grasps whose quaternions are on the positive hemi-hypersphere.

$$G_1 \subset G_D = \{g_i | q_{iw} \geq 0\}$$

Secondly, the sampled grasp positions' height must be greater than the sum of the height of the desk and the radius of the gripper. Any grasp below that threshold would crash into the table when attempting to grasp it.

$$G_2 \subset G_1 = \{g_i | z_i > z_{desk} + r_{gr}\}$$

Thirdly, grasps were restricted to lie within the line of sight of the camera to avoid problems related to occlusion. This was recreated by sampling from a cone with origin at the object's center and an axis given by the vector $\mathbf{a}$ from the camera frame to the object frame.

$$G_3 \subset G_2 = \{g_i | \frac{p_i}{|p_i|} \cdot \mathbf{a} > \cos(70)\}$$

The final restriction is drawn from the fact that we're using an antipodal gripper. This gives rise to the problem that for any valid grasp with an arbitrary position and orientation, there exists a second valid grasp with the same position but with a rotation of 180° along the z axis of the original grasp.

In order to reduce the ambiguity resulting from this, we sample the grasps where the orientation of the gripper is "pointing upwards". That is, that the x component of the reference frame of the gripper base is positive.

25.6.2 Manually generated dataset

Initially, we constructed a testing dataset by manually taking samples of grasps in the simulated environment by carefully placing the gripper at a suitable pose to grasp the objects and then saving the poses $g = \{p, q\}$ relative to the objects' center. Then we would randomly place the object on the table and sample the saved grasps that satisfied the geometric constraints in 25.6.1 and save them with their associated point clouds into pickled files. Despite being faster than real-life data collection, we realized that this method was still too time-consuming and the dataset was taking too much space. Because of this, we decided to drop this method and look for alternatives.

25.6.3 Sampling based dataset

For the second dataset, we considered using or adapting an already existing dataset that was publicly available. We considered many options, like [824] and [834] which are datasets of RGB-D images with labeled grasps suited for training our model. In the end, we opted for the 'A billion ways to grasp' dataset [835] which is a dataset of successful grasp poses relative to the center of mass of 21 objects from the YCB object set. We chose this dataset because

the labeled grasp poses are formatted as the x, y, z position and the orientation as unit quaternions, so it already had the desired format of the input and outputs of our model. Additionally, since the grasps are annotated relative to the center of the object, adapting this dataset to our existing dataset generator was simple.

Similarly to the manually generated dataset, we would randomly place one of the objects from the object pool onto the table, then randomly sample 1000 grasps from the A billion ways to grasp dataset for the corresponding object. Then we transformed the grasps to the reference frame of the object and applied the constraints in 25.6.1. After preprocessing, 10 random successful grasps were saved along with their associated point cloud. To improve the saving and sampling speed for training, we saved the data into an SQL database.

25.7 Experiments

To evaluate the performance of the network, two tests were designed to measure its speed and accuracy. The first test samples how long it takes for the system to output a grasp proposal once given a valid point cloud. That is, we do not take into consideration preprocessing time. The second test is designed to measure the accuracy of the system, wherein we perform 200 grasps per object placed randomly on the testing environment and evaluate the percentage of successful grasps.

25.7.1 Testbed

A simulated testing environment was built using ROS-Gazebo to test the accuracy and response time of the proposed grasp from the system.

Similar to the training environment, the test world consists of a simple desk with a single object placed on top with a randomized position and orientation and a service robot placed directly in front of it with its spatial movement locked. The robot captures a point cloud centered around the object and crops it to the network's input size. Once the point cloud is preprocessed, a timer starts and the input is fed into the network. Once the system finishes calculating the grasp proposal the timer stops and the system response time is captured.

The grasp proposal is evaluated by placing the robot gripper at the predicted pose with the fingers open. Once the gripper is placed at the specified position and orientation, the gripper closes its fingers. Two collision sensors check for contact during this phase. If at the end of the grasping phase both sensors detect contact with the object, i.e., both fingers are still touching the

object, the grasp is considered successful. Otherwise, it is considered a failed grasp.

This test is repeated 200 times for each object in the training pool and for an additional 4 objects that were not in the original training dataset. The accuracy is calculated as the percentage of successful grasps over all grasp attempts. The results are shown in the following section.

25.8 Results

For these experiments we only tested the network with the kernel mixture model approach. During development and initial testing we realized that the geodesic regression approach had poor performance compared to the kernel mixture approach. Future improvements to this method have been analyzed, like that of Geodesic Bin and delta proposed in [836]. However, this is a work in progress and thus not included in this round of experiments.

After running the experiments with the kernel mixture model we observed that the network had an average response time of 0.134s. The average grasp accuracy per object, and per object shape are shown in the following tables:

25.9 Conclusion

We trained a network that is capable of approximating the probability distribution of suitable grasp proposals directly from a point cloud and sampling from it with reasonable accuracy for most objects.

We observed that the network achieved the best performance when grasping larger symmetrical objects. Perhaps because the cylindrical and rectangular box geometries are overrepresented in the training data (almost half of all objects). For smaller objects with complicated geometries, the network struggled to find appropriate grasp proposals. Likely due to the fact that grasping these objects requires a higher degree of precision and be calibrated to the characteristics of the gripper.

When comparing to the current implemented system detailed in [837] we found that the network has similar albeit slightly worse accuracy. However, it's important to note that the current system requires robust image processing techniques like object segmentation, contour detection and filtering since this method is very sensitive to outliers and noise. In comparison, the neural network model can achieve similar performance without these methods since it only requires a bounding box around the object and it is not necessary to eliminate the background.

TABLE 25.1

Grasp accuracy per object

YCB Object	Object Shape	Grasp Accuracy[%]
001_chips_can	Cylindrical	79.5
002_master_chef_can	Cylindrical	71
003_cracker_box	Large box	70.5
004_sugar_box	Large box	64.5
005_tomato_soup_can	Cylindrical	69.5
006_mustard_bottle	Flat	69
007_tuna_fish_can	Cylindrical	59
008_pudding_box	Small box	67.5
009_gelatin_box	Small box	72.5
010_potted_meat_can	Small box	68.5
011_banana	Other	53
019_pitcher_base	Cylindrical	51.5
021_bleach_cleanser	Flat	44
024_bowl	Other	62.5
025_mug	Cylindrical	58
035_power_drill	Other	26
036_wood_block	Large box	78
037_scissors	Tool	22.5
040_large_marker	Tool	21.5
048_hammer	Tool	9
051_large_clamp	Tool	11.5
052_extra_large_clamp	Tool	24
056_tennis_ball	Spherical	60
061_foam_brick	Small box	67.5
077_rubiks_cube	Cubic	70.5

TABLE 25.2

Grasp accuracy per object shape

Object Shape	Grasp Accuracy[%]
Cylindrical	64.75
Spherical	60
Large box	71
Small box	69
Cubic	70.5
Flat	56.5
Tool	17.7
Other	47.166

Future work will be focused on improving the performance of the network by training with better quality data. Primarily by improving the range and distribution of geometries in the training dataset, and increasing the volume of data. Furthermore, we seek to implement and test the system for real life grasping. Using domain adaptation methods and Bayesian inference to bridge the gap between the simulated and real environments.

Acknowledgments. This work supported by UNAM-DGAPA under grant IT102424.